道の採集家 技術選定・開発方針まとめ

ハッカソン Androidアプリ版 ／ takalabo ／ 2026-11-06 v2（試作で数値は調整する前提）

1. 前提条件

企画	歩いた軌跡を積み木パーツに変換し、バランスを取りながらタワーを積む。「次に欲しい形を拾いに行く」ことが外出の動機になる（企画書「神林 アイディア案」より）。
提出形式	**Androidネイティブ必須**（WebView包み不可）。
実行基盤	App Engine / Compute Engine / GKE / **Cloud Run** / Firebase App Hosting / Firebase Hosting の**いずれかを必ず使う**。
加点	Google技術の利用で加点（必須ではない）。
期間・人数	3〜5日、4人。チームの得意技術は Kotlin / Android Studio。
予算	Google Cloud クレジット **$300**。請求先アカウントを有効化できるので、通常の Maps API キー・Vertex AI・Cloud Run の最小インスタンスが使える。5 日間の想定使用額は数ドル。
進め方	**採集（GPS→パーツ）を先行**。積み上げゲームの細部と対戦方式は、Day2夕方に試作を触って決める。

2. 技術構成（決定）

企画書のWeb構成（React / Matter.js / Firebase Hosting 等）は全面的に置き換える。

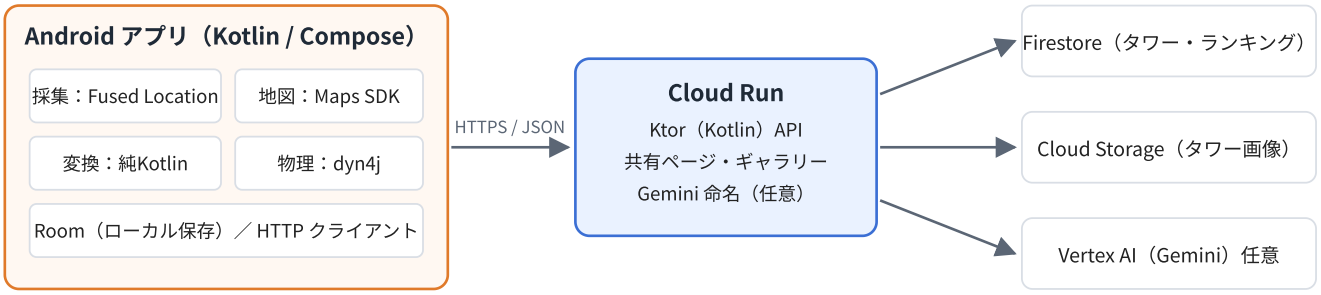
領域	採用	理由・補足
アプリ UI	Kotlin ＋ Jetpack Compose	チームの得意分野。画面はすべてCompose。minSdk 26、targetSdk は最新。
2D物理	**dyn4j** 6.0.0（`org.dyn4j:dyn4j`）＋ Compose Canvas で描画	純Javaの物理ライブラリで依存が1つ。1ボディに複数fixtureを持てるので凹型パーツ（L・U・輪）を分解せずに扱える。ゲームエンジン経験者がいれば libGDX＋Box2D でも可。
位置情報	Google Fused Location Provider（play-services-location）＋ フォアグラウンドサービス（type=location）	画面ロック中も記録できる＝ネイティブにする最大の理由。`ACCESS_BACKGROUND_LOCATION` は使わない（アプリを開いた状態でサービスを開始すれば不要）。
地図	Google Maps SDK for Android ＋ maps-compose	モバイルSDKは無制限無料（クレジットは減らない）。請求先アカウントを有効化した**通常の API キー**を使う。Demo Key の 1 日上限を気にしなくてよい。
ローカル保存	Room	手持ちパーツ・生軌跡・タワー配置。ゲームの核はオフラインで動かす。
通信	Retrofit または Ktor Client（JSON）	アプリ側に Firebase SDK は入れない。普通のHTTPだけ。

サーバ	Google **Cloud Run** ＋ Ktor（Kotlin）	実行基盤の条件を満たす本命。Jib で Artifact Registry にイメージを送ってデプロイ（手元に Docker 不要）。無料枠内、使わない時はゼロにスケール。
サーバ側保存	Google Firestore ＋ Cloud Storage	Cloud Run のサービスアカウントで接続。鍵ファイルを扱わない。
生成AI（任意）	Google Vertex AI（Gemini）をサーバ経由	パーツの命名・一言、次に拾う形の提案。ゲーム成立には不要。APIキーがアプリに入らない。Vertex AI は請求先アカウントが必要なので、クレジットがある今回は問題なく使える。
共有	Android 共有シート ＋ サーバの共有ページ（OGP画像付き）	LINE / X に貼るとタワー画像がプレビューされる。

使わないもの

不採用 React＋TypeScript、Matter.js、Firebase Hosting、アプリ内 Firebase SDK（Auth / Firestore / Storage / AI Logic）、`ACCESS_BACKGROUND_LOCATION`、リアルタイムのオンライン対戦（WebSocket 等）。

3. システム構成



原則：サーバが止まっても積み上げゲームと採集はアプリ単体で動く。サーバは「共有・ギャラリー・命名」など、デモで映るが核ではない機能に限定する。

4. サーバ API（Cloud Run）

エンドポイント	用途	目標日
`GET /`	Hello World。**Day1 に必ずデプロイ**して実行基盤の条件を初日に満たす。	Day1
`POST /towers`	タワー（パーツ形状・配置・画像）を保存し、共有URLを返す。位置情報は含めない。	Day2
`GET /towers/{id}`	共有ページ（HTML、OGP画像付き）。LINE / X でプレビューが出る。	Day3
`GET /towers`	最近のタワー・高さ順ランキング（アプリ内「みんなのタワー」画面）。	Day3
`POST /parts/name`	形を送ると Vertex AI（Gemini）が名前と一言を返す。失敗時はアプリ側のデフォルト名。	Day4
`POST /challenges` ほか	非同期スコアバトルを採用した場合のみ（同じパーツセットの配布と結果記録）。	Day4（要決定）

5. パーツの定義（チーム共通のデータ形式・Day1 朝に確定）

採集側はこれを**作る**、ゲーム側はこれから物理ボディを**作る**、サーバはこれをJSONで**受け取る**。この境界を固定しておけば、ゲームのルールや対戦方式が後で変わっても採集側は影響を受けない。

```
data class Part(
    val id: String,
    val points: List<Pt>,     // 正規化済み。長辺が 1.0 に収まる座標。頂点は最大 12 個
    val thickness: Float,     // 太さ（長辺に対する比率。初期値 0.15）
    val closed: Boolean,      // 輪なら true（始点と終点をつなぐ）
    val kind: Kind,           // BAR / L / U / RING / ZIGZAG / OTHER（表示用の分類）
    val distanceM: Int,       // 歩いた距離 [m]
    val walkedAt: Long,       // 採集日時（epoch millis）
    val name: String? = null  // Gemini 命名（任意、後から入る）
)

data class Pt(val x: Float, val y: Float)
```

- 生のGPS軌跡は `Part` に含めず、別テーブル（デバッグ用）に保存する。共有・サーバ送信には出さない。
- フィールドを変えるときは必ず全員に周知する。

6. 採集・変換の初期パラメータ

項目	初期値
記録の開始・停止	手動ボタン
最小距離	200 m 未満はパーツにしない
自動停止	2 時間

項目	初期値
座標変換	開始点基準の等距円筒近似でメートルへ
簡略化	Douglas-Peucker、ε = 15 m
頂点上限	12（超えたら ε を上げて再簡略化）

精度フィルタ	accuracy > 30 m の点を捨てる
最小移動	前回点から 5 m 未満は無視
更新間隔	2〜3 秒、高精度モード

正規化	縦横比を保ち長辺 = 1.0
輪の判定	終点が始点から長辺の 10% 以内
分類	曲がり角の数と閉路で 棒 / L / U / 輪 / ジグザグ

GPX / JSON の軌跡ファイルを読み込む「再生モード」を必ず作る（審査員は歩けない）。Android エミュレータの拡張設定からも GPX 再生ができる。

7. 物理・積み上げゲームの初期値（試作で調整）

項目	方針
ボディの作り方	簡略化した線分 1 本ごとに細長い矩形（関節に小さい円）を作り、1 ボディに複数 fixture として合成。凹分解が不要で、輪の内側の穴もそのまま残る。
物理パラメータ	摩擦 0.7、反発 0、固定タイムステップ 1/60 秒、軽い角減衰。
静止判定	全ボディが約 1 秒静止（dyn4j のスリープ判定）したら次のパーツへ。
台	幅はパーツ長辺の 1.2 倍。台の外に落ちたパーツは消える（手持ちからは失われない）。
操作	横ドラッグで位置、15° 刻みの回転ボタン、「置く」で落下。カメラはタワーの高さに合わせて自動スクロール。
スコア	5 個を置き切って静止した時点の、台の上面から最高点までの高さ。全部落ちたら 0。
保存	配置したパーツの ID・位置・角度を保存（物理の再現ではなく最終配置）。
初回サンプル	棒・L・U・輪・ジグザグの 5 種類を同梱。歩かなくても遊べる状態を Day1 に作る。

8. 対戦の候補（Day2 夕方に決定）

方式	概要	コスト	備考
交互置き（1 台）	共有の手持ちから交互に置き、崩した方が負け。	低	サーバ不要。審査員 2 人に対戦してもらえる。デモ向き。
非同期スコアバトル	同じパーツセット（「今日の 5 個」）で高さを競い、Cloud Run に記録してランキング表示。	中	REST の追加だけで済む。
リアルタイム対戦	WebSocket / リスナーで同時進行。	高	5 日では他を削ることになる。今回は見送り推奨。

どの方式でも「置く・落ちる・静止・高さ」の一人用コアは共通。ゲーム担当はまずそこを作り、対戦の決定を待たない。

9. 分担とスケジュール

	A：GPS採集	B：軌跡→パーツ変換	C：サーバ	D：積み上げ試作
Day1	権限、Fused Location、フォアグラウンドサービスの骨組み	Douglas-Peucker・正規化・分類を純 Kotlin で、ユニットテスト付き	Cloud Run に Hello World をデプロイ。Firestore / Storage 準備	dyn4j ＋ Compose でサンプル 5 個を積める状態に
Day2	地図に軌跡をリアルタイム描画。開始／停止で生軌跡を保存	軌跡→Part を通しで動かす。手持ち一覧画面。GPX 再生モード	POST /towers、Cloud Storage への画像保存	操作・静止判定・高さ。**夕方に全員で試作を触り、ルールと対戦方式を決める**
Day3	**A＋B で実際に外を歩いて通しテスト**。精度閾値・最小移動・ε の調整。Part の見た目（形の描画）を仕上げ		共有ページ（OGP）、GET /towers ギャラリー	決めたルールを実装
Day4	採集側の磨き、対戦 UI の手伝い	同左	Gemini 命名、対戦に必要な API	対戦モード
Day5	予備・デモ準備・発表資料。デモ当日は Cloud Run の最小インスタンスを 1 にしてコールドスタートを避ける			

Day1 朝に全員で「パーツの定義」（5 章）とリポジトリ構成（`:app` / `:core-model` / `:collect` / `:game` / `server/`）を決めてから分かれる。

10. 設定・権限のチェックリスト（詰まりやすい点）

項目	内容
Manifest（位置情報）	`ACCESS_FINE_LOCATION`、`FOREGROUND_SERVICE`、`FOREGROUND_SERVICE_LOCATION`（Android 14 以降は無いと起動時に例外）。サービスに `android:foregroundServiceType="location"`。
通知権限	Android 13 以降は `POST_NOTIFICATIONS` を実行時に要求（無くても記録はできるが「記録中」通知が出ない）。
サービスの開始	必ずアプリを開いた状態（画面上のボタン）から開始する。バックグラウンドからの開始は制限される。
Maps	請求先アカウントを有効化した通常の API キーを使う。キーは Android アプリのパッケージ名と署名（SHA-1）で制限し、リポジトリに直接コミットしない（`local.properties` ＋ secrets-gradle-plugin など）。
Cloud Run	Jib でデプロイ。Firestore / Storage / Vertex AI の権限は Cloud Run のサービスアカウントに付与。鍵ファイルは作らない。
予算管理	Billing で予算アラート（例：$50）を 1 つ設定。終了後は Cloud Run の最小インスタンスを 0 に戻し、不要なリソースを削除。クレジットが無料トライアル由来なら有効期限（通常 90 日）を確認。
オフライン動作	採集・変換・積み上げはサーバ無しで完結させる。サーバ呼び出しは失敗しても UI が止まらないようにする。

11. 要確認・未決事項

項目	状況
レギュレーションに「AI 製品必須」の一文があるか	実行基盤の指定が Google Cloud ハッカソンの形式と同じなので要確認。ある場合は Vertex AI 経由の Gemini 命名で同時に満たす。
対戦方式	Day2 夕方、試作を触って決定（8 章）。
パーツサイズと距離の関係	まずは全パーツ同サイズ。「長く歩く動機」を足す場合はサイズではなく太さを距離に連動させる案。
クレジットの適用先と有効期限	$300 クレジットをどの請求先アカウント・プロジェクトに紐づけるか、期限はいつまでかを Day1 に確認。
個数・サイズなどの数値	企画書どおり、試作を遊んで調整する。